

Access Modifiers in Python

Public, protected and private members explained
with clean programs and real-world examples.

THEORY • PROGRAMS • OUTPUTS • INTERVIEW PREPARATION

Participant Training Guide

Thaneshwara M

01 Introduction

Access modifiers control how variables and methods of a class are intended to be accessed. Python supports three access levels through naming conventions.

PUBLIC

Written normally: `name`
Accessible from anywhere.

PROTECTED

Single underscore: `_name`
Intended for class and child classes.

PRIVATE

Double underscore: `__name`
Intended for use inside the same class.

Important Python behaviour

Unlike Java or C#, Python does not enforce strict public, protected and private keywords. It uses naming conventions and name mangling. Responsible developers follow these conventions.

Quick syntax

```
self.name      # Public
self._salary   # Protected
self.__balance # Private
```

02 Public access modifier

Public members are accessible inside the class, inside child classes and outside the class. By default, Python variables and methods are public.

Student example

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

student = Student("Rahul", 85)

print(student.name)
print(student.marks)
student.display()
```

```
OUTPUT
Rahul
85
Name: Rahul
Marks: 85
```

Understanding

name, marks and display() are public. The object accesses them directly using the dot operator.

03 Protected access modifier

A protected member starts with a single underscore. It is intended for use inside the class and its child classes.

```
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self._salary = salary

    def display_salary(self):
        print("Salary:", self._salary)

class Manager(Employee):

    def show_details(self):
        print("Manager Name:", self.name)
        print("Manager Salary:", self._salary)

manager = Manager("Anil", 80000)
manager.show_details()
```

OUTPUT

```
Manager Name: Anil
Manager Salary: 80000
```

Can we access it outside?

```
print(manager._salary) # Technically possible
```

Python allows this, but the underscore communicates: "This member is for internal use; access it carefully." Outside access is discouraged.

04 Private access modifier

A private member starts with two underscores. Python changes its internal name using name mangling, preventing normal direct access from outside.

Bank account example

```
class BankAccount:

    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Amount deposited successfully.")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount.")
        elif amount <= self.__balance:
            self.__balance -= amount
            print("Withdrawal successful.")
        else:
            print("Insufficient balance.")
```

04 Private access • Continued

```
def check_balance(self):  
    print("Current Balance:", self.__balance)  
  
account = BankAccount(1001, 50000)  
  
account.deposit(5000)  
account.withdraw(10000)  
account.check_balance()
```

OUTPUT

Amount deposited successfully.
Withdrawal successful.
Current Balance: 45000

Direct access

```
print(account.__balance)
```

OUTPUT

AttributeError: 'BankAccount' object has no
attribute '__balance'

Why this design is useful

The balance cannot be safely changed directly. It is modified through `deposit()` and `withdraw()`, where business validation is applied.

05 Name mangling

Python transforms a double-underscore member internally. For example, `__balance` inside `BankAccount` becomes approximately `_BankAccount__balance`.

Purpose

Avoid accidental access and accidental overriding.

Important

It is not absolute security. It is controlled internal naming.

```
class Demo:

    def __init__(self):
        self.__value = 100

obj = Demo()

print(obj.__value)          # Error
print(obj._Demo__value)    # Possible, not recommended
```

OUTPUT
AttributeError for obj.__value
100 for obj._Demo__value

Trainer note

Explain to participants that private in Python means name-mangled and intended for internal use. It does not provide unbreakable data security.

06 Getter and setter methods

Getter and setter methods provide controlled access to private variables. The getter reads a value; the setter validates and updates it.

```
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
            print("Salary updated successfully.")
        else:
            print("Salary must be greater than zero.")

employee = Employee("Rahul", 50000)

print("Current Salary:", employee.get_salary())
employee.set_salary(60000)
print("Updated Salary:", employee.get_salary())
```

OUTPUT

```
Current Salary: 50000
Salary updated successfully.
Updated Salary: 60000
```

07 Using @property

@property is the Pythonic way to provide controlled access while keeping clean attribute-style syntax.

```
class Product:

    def __init__(self, name, price):
        self.name = name
        self.__price = price

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, value):
        if value > 0:
            self.__price = value
            print("Price updated successfully.")
        else:
            print("Price must be greater than zero.")

product = Product("Laptop", 50000)

print("Current Price:", product.price)
product.price = 55000
print("Updated Price:", product.price)
```

OUTPUT

```
Current Price: 50000
Price updated successfully.
Updated Price: 55000
```

08 Complete comparison

Use this table for quick classroom revision.

TYPE	SYNTAX	SAME CLASS	CHILD CLASS	OUTSIDE
Public	name	Yes	Yes	Yes
Protected	<code>_name</code>	Yes	Yes	Possible, discouraged
Private	<code>__name</code>	Yes	Not directly	Not directly

Easy memory formula

No underscore

Public: available everywhere.

One underscore

Protected: internal and inheritance use.

Two underscores

Private: same-class use through name mangling.

Access modifier vs encapsulation

Access conventions support encapsulation. Encapsulation groups data and behaviour and controls how internal data is read or changed.

09 Participant practice

Task 1 • Student

Create public name and marks variables. Display them outside the class.

Task 2 • Employee

Create a protected `_salary` variable and access it inside a Manager child class.

Task 3 • Bank

Create a private `__balance` variable. Add `deposit()`, `withdraw()` and `check_balance()`.

Task 4 • Product

Create a private `__price` variable. Validate updates using `@property`.

Common mistakes

- Assuming a single underscore blocks outside access.
- Treating double underscores as absolute security.
- Modifying sensitive data without validation.
- Forgetting that Python uses conventions instead of access keywords.

Discussion question

Why is `account.deposit(5000)` better than changing the balance directly? Discuss validation, consistency and security.

10 Interview-ready answer

“Python supports public, protected and private-style members through naming conventions. Public members have no underscore, protected members use a single underscore, and private members use double underscores. Private members use name mangling and should be accessed through controlled methods or properties.”

Final revision

- Public: directly accessible.
- Protected: internal-use convention.
- Private: double underscore and name mangling.
- Getter/setter: controlled method access.
- @property: clean, Pythonic controlled access.

TRAIN • PRACTICE • BUILD

Prepared by Thaneshwara M

Python OOP Participant Training Series